Opisanie funkcij szlazhivanija

(Python, programmistskaja versija)

1. Boxcar

Signatura

```
def boxcar(win: int, data: list) -> list
```

Parametry

Parametr	Tip	Opisanie
win	int	Razmer okna usrednenija
data	list[float]	Vhodnoj signal dlinoj N

Algoritm

```
def boxcar(win, data):
    half = win // 2
    n = len(data)
    avg = []
    for i in range(n):
        start = max(0, i + 1 - half)
        end = min(half + i, n)
        sum_array = sum(data[start:end])
        aver = sum_array / len(data[start:end])
        avg.append(aver)
    return avg
```

Dlja kazhdogo indeksa i vycisljajutsja granicy okna [start, end) s obrezkoj po krajam massiva. Vychisljaetsja arifmeticheskoe srednee po srezu data[start:end].

Slozhnost'

Parametr	Znachenie
Vremja	O(N · win)
Pamjat'	O(N)
Granic. uslovija	Clamping (obrezka okna)

2. Gaussian

Signatura

```
def gaussian(data: list, win: int) -> list
```

Algoritm — jadro

```
def gaussian(data, win):
    half = win // 2
    sigma = win / 6.0
    n = len(data)
    k = [math.exp(-0.5*(i - half)**2 / sigma**2)
         for i in range(win)]
    k = [x/sum(k) for x in k]
```

Algoritm — dopolnenie i svjortka

```
    p = ([data[i] for i in range(half-1, -1, -1)]
         + data
         + [data[i] for i in range(n-1, n-half-1, -1)])
    return [sum(k[j]*p[i+j] for j in range(win))
            for i in range(n)]
```

Jadro Gaussa: w[k] = exp(-0.5*(k-half)² / sigma²), sigma = win/6. Normalizacija: w_norm[k] = w[k] / sum(w). Dopolnenie: simmetricheskoe otrazhenie na half elementov. Svjortka: kazhdyj vyhodnoj otsc'jot — vveshennaja summa po jadru.

Slozhnost'

Parametr	Znachenie
Vremja	O(N · win)
Pamjat'	O(N + win)
Granic. uslovija	Mirror reflection (otrazhenie)

3. Progressive

Signatura

```
def progressive(data: list) -> list
```

Algoritm

```
def progressive(data):
    result = []
    total = 0.0
    for i, value in enumerate(data):
        total += value
        result.append(total / (i + 1))
    return result
```

Nakaplivaetsja suma total. Na kazhom shage i vycisljaetsja total / (i + 1) — srednee otsc'jotov ot 0 do i.

Slozhnost'

Parametr	Znachenie
Vremja	O(N)
Pamjat'	O(N)
Granic. uslovija	Net (okno rastot ot 1 do N)

4. Hybrid

Signatura

```
def hybrid(data: list, win: int) -> list
```

Algoritm

```
def hybrid(data, win):
    n = len(data)
    box = boxcar(win, data)
    prog = progressive(data)

    out = []
    for i in range(n):
        if i < win:
            out.append(box[i])
        elif i < 2 * win:
            z = (i - win) / win
            k = (1.0 - z) * box[i] + z * prog[i]
            out.append(k)
        else:
            out.append(prog[i])
    return out
```

Logika

Uslovie	Dejstvie
i < win	Beretsja boxcar[i]
win <= i < 2*win	Linejnaja interpoljacija: (1-z)*box + z*prog
i >= 2*win	Beretsja progressive[i]

Koefficient interpoljacii: z = (i - win) / win, z [0, 1).

Slozhnost'

Parametr	Znachenie
Vremja	O(N · win) (iz-za boxcar)
Pamjat'	O(N)
Granic. uslovija	Zavisit ot boxcar (clamping)